

This notebook was ran locally and would need adjustments to the file paths to reproduce the results (Google Drive must be set up and the paths to the models should be adjusted)

```
# Google drive set up
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

```
pip install ultralytics
```

 [Show hidden output](#)

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision.models import resnet50, ResNet50_Weights
import numpy as np
import cv2
from PIL import Image
import torchvision.transforms as transforms
from typing import List, Tuple, Dict, Optional
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from ultralytics import YOLO
import matplotlib.patches as patches
import os
from tiffiffle import imread
import json
```

 Creating new Ultralytics Settings v0.0.6 file   
 View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'  
 Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs\_dir=path/to/dir'. For help see <https://docs.ultralytics.com/quickstart/#ultralytics-settings>.

```
class YOLOBuildingDamageAssessmentPipeline:
    def __init__(self, yolo_model_path: str, classification_model, device='cuda',
                 confidence_threshold=0.5, # YOLO confidence threshold
                 crop_size=224):          # Size for classification input

        # Load YOLO model
        self.yolo_model = YOLO(yolo_model_path) # Path to trained YOLO Model
        self.classification_model = classification_model.to(device) # Load classification model
        self.device = device # Device to run inference on
        self.confidence_threshold = confidence_threshold # Confidence to set for YOLO Detection
        self.crop_size = crop_size # Building crops size for classification (set to 224)

        # Set classification model to evaluation mode
        self.classification_model.eval()

        # Normalization for classification (ImageNET)
        self.cls_normalize = transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225])
```

```

        self.crop_size = [0.225, 0.225, 0.225]
    )

    # Damage class namesy
    self.damage_classes = ['no-damage', 'minor-damage', 'major-damage', 'destroyed']

# Convert .tif image to .png format using OpenCV (similar to the function used in training)
# Maintains same dimensions for coordinate consistency
def convert_tif_to_png_for_yolo(self, tif_image: np.ndarray) -> np.ndarray:
    try:
        # OpenCV conversion (matches your convert_with_opencv)
        if len(tif_image.shape) == 3 and tif_image.shape[2] == 3:
            # Convert RGB to BGR for OpenCV compatibility, then back to RGB
            bgr_image = cv2.cvtColor(tif_image, cv2.COLOR_RGB2BGR)
            png_image = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2RGB)
        else:
            png_image = tif_image.copy()

        # Ensure uint8 format
        if png_image.dtype != np.uint8:
            if png_image.max() <= 1.0:
                png_image = (png_image * 255).astype(np.uint8)
            else:
                png_image = png_image.astype(np.uint8)

        return png_image

    except Exception as e:
        # Fallback
        print(f"Warning: OpenCV conversion failed, using fallback: {e}")
        if tif_image.dtype != np.uint8:
            if tif_image.max() <= 1.0:
                return (tif_image * 255).astype(np.uint8)
            else:
                return tif_image.astype(np.uint8)
        return tif_image.copy()

# Preprocess building crop for classification
def preprocess_crop_for_classification(self, crop: np.ndarray) -> torch.Tensor:
    # Convert to PIL and resize
    if isinstance(crop, np.ndarray):
        if crop.dtype != np.uint8:
            crop = (crop * 255).astype(np.uint8)
        crop = Image.fromarray(crop)
    # Transform for classification
    transform = transforms.Compose([
        transforms.Resize((self.crop_size, self.crop_size)),
        transforms.ToTensor(),
        self.cls_normalize
    ])
    return transform(crop).unsqueeze(0)

def detect_buildings_yolo(self, image: np.ndarray, detection_image: np.ndarray = None) -> List[Dict]:
    """
    Stage 1: Detect buildings using YOLO model
    """

```

```

image: Input image for detection (can be .png converted version)
detection_image: Optional separate image for coordinate reference (original .tif)
Returns: List of building detection dictionaries with coordinates adjusted for detection_image if provided
"""

# Use detection_image for coordinate reference if provided
reference_image = detection_image if detection_image is not None else image

# Run YOLO inference
results = self.yolo_model(image, conf=self.confidence_threshold)

# Calculate scaling factors if images have different dimensions
scale_x = scale_y = 1.0
if detection_image is not None:
    scale_x = reference_image.shape[1] / image.shape[1] # width scaling
    scale_y = reference_image.shape[0] / image.shape[0] # height scaling

building_detections = []

# Process YOLO results
for result in results:
    boxes = result.boxes
    if boxes is not None:
        for i, box in enumerate(boxes):
            # Get bounding box coordinates (xyxy format)
            x1, y1, x2, y2 = box.xyxy[0].cpu().numpy()
            confidence = box.conf[0].cpu().numpy()

            # Scale coordinates to reference image if needed
            x1, y1, x2, y2 = x1 * scale_x, y1 * scale_y, x2 * scale_x, y2 * scale_y

            # Convert to integers
            x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

            # Ensure coordinates are within reference image bounds
            x1 = max(0, min(x1, reference_image.shape[1]-1))
            y1 = max(0, min(y1, reference_image.shape[0]-1))
            x2 = max(x1+1, min(x2, reference_image.shape[1]))
            y2 = max(y1+1, min(y2, reference_image.shape[0]))

            # Calculate building properties
            width = x2 - x1
            height = y2 - y1
            area = width * height
            building_size = max(width, height)
            centroid_x = (x1 + x2) / 2
            centroid_y = (y1 + y2) / 2

            # Store building detection
            building_info = {
                'id': i,
                'bbox': (x1, y1, x2, y2),
                'confidence': float(confidence),
                'area': area,
                'building_size': building_size,
                'width': width,
                'height': height,
                'centroid': (centroid_x, centroid_y)
            }

```

```

    }

    building_detections.append(building_info)

return building_detections

def extract_building_crops_from_detections(self, image: np.ndarray,
                                          building_detections: List[Dict]) -> List[Dict]:
    """
    Cropping stage: Extract building crops from YOLO detections
    Use similar cropping logic as training to ensure consistency
    image: Original image
    building_detections: List of YOLO detection dictionaries
    Returns: Updated building detection dictionaries with crops
    """
    for building in building_detections:
        x1, y1, x2, y2 = building['bbox']
        width = building['width']
        height = building['height']
        building_size = building['building_size']

        # Calculate adaptive padding based on building size (matching training approach)
        padding_ratio = 0.3 # Same as training
        padding = max(16, int(building_size * padding_ratio))

        # Calculate crop boundaries with padding
        crop_min_x = max(0, x1 - padding)
        crop_min_y = max(0, y1 - padding)
        crop_max_x = min(image.shape[1], x2 + padding)
        crop_max_y = min(image.shape[0], y2 + padding)

        # Final crop dimensions
        crop_width = crop_max_x - crop_min_x
        crop_height = crop_max_y - crop_min_y
        crop_size = max(crop_width, crop_height)

        # Handle large crops (matching training max_crop_size logic)
        max_crop_size = 512
        if crop_size > max_crop_size:
            # Calculate reduced padding
            excess = crop_size - max_crop_size
            reduced_padding = max(8, padding - (excess // 2))

            # Recalculate with reduced padding
            crop_min_x = max(0, x1 - reduced_padding)
            crop_min_y = max(0, y1 - reduced_padding)
            crop_max_x = min(image.shape[1], x2 + reduced_padding)
            crop_max_y = min(image.shape[0], y2 + reduced_padding)

            crop_width = crop_max_x - crop_min_x
            crop_height = crop_max_y - crop_min_y

        # If still too large, center crop around building centroid
        if max(crop_width, crop_height) > max_crop_size:
            centroid_x, centroid_y = building['centroid']

            half_size = max(crop_width, crop_height) // 2

```

```

        crop_min_x = max(0, int(centroid_x - half_size))
        crop_min_y = max(0, int(centroid_y - half_size))
        crop_max_x = min(image.shape[1], crop_min_x + max_crop_size)
        crop_max_y = min(image.shape[0], crop_min_y + max_crop_size)

        # Adjust if crop goes outside bounds
        if crop_max_x - crop_min_x < max_crop_size:
            crop_min_x = max(0, crop_max_x - max_crop_size)
        if crop_max_y - crop_min_y < max_crop_size:
            crop_min_y = max(0, crop_max_y - max_crop_size)

        # Update final dimensions
        crop_width = crop_max_x - crop_min_x
        crop_height = crop_max_y - crop_min_y
        crop_size = max(crop_width, crop_height)

        # Calculate building-to-crop ratio (similar to training)
        building_to_crop_ratio = building_size / crop_size

        # Extract crop
        crop = image[crop_min_y:crop_max_y, crop_min_x:crop_max_x]

        # Update building information with crop data
        building.update({
            'crop': crop,
            'crop_bbox': (crop_min_x, crop_min_y, crop_max_x, crop_max_y),
            'crop_size': crop_size,
            'building_ratio': building_to_crop_ratio,
            'padding_used': padding
        })

    return building_detections

def classify_damage(self, building_detections: List[Dict]) -> List[Dict]:
    """
    Stage 2: Classification of damage level for each building crop
    building_detections: List of building detection dictionaries with crops
    Returns: Building detections with damage predictions added
    """
    with torch.no_grad():
        for building in building_detections:
            # Preprocess crop
            input_tensor = self.preprocess_crop_for_classification(
                building['crop']).to(self.device)

            # Get classification output
            output = self.classification_model(input_tensor)

            # Get probabilities and prediction
            probabilities = F.softmax(output, dim=1)
            predicted_class = torch.argmax(probabilities, dim=1).item()
            confidence = probabilities[0, predicted_class].item()

            # Add damage information to building
            building['damage_class'] = self.damage_classes[predicted_class]
            building['damage_confidence'] = confidence
            building['damage_probabilities'] = probabilities[0].cpu().numpy()

```

```

        building['damage_probabilities'] = probabilities[0].cpu().numpy())

    return building_detections

# Full pipeline: detect buildings with YOLO and classify damage
def process_image(self, tif_image: np.ndarray, png_image: np.ndarray = None) -> Dict:
    # Convert .tif to .png for YOLO if not provided
    if png_image is None:
        print("Converting .tif to .png for YOLO detection...")
        png_image = self.convert_tif_to_png_for_yolo(tif_image)

    # Verify dimensions match (crucial for coordinate scaling)
    if png_image.shape[:2] != tif_image.shape[:2]:
        print(f"Warning: Dimension mismatch! TIF: {tif_image.shape[:2]}, PNG: {png_image.shape[:2]}")
        print("Coordinates may not align properly.")

    print("Stage 1: Building detection using YOLO on .png")
    building_detections = self.detect_buildings_yolo(png_image, tif_image)

    print(f"Extracting crops for {len(building_detections)} detected buildings from original .tif")
    building_detections = self.extract_building_crops_from_detections(tif_image, building_detections)

    print(f"Stage 2: Classifying damage for {len(building_detections)} buildings")
    building_detections = self.classify_damage(building_detections)

    # Compile results
    results = {
        'original_tif_image': tif_image,
        'png_detection_image': png_image,
        'buildings': building_detections,
        'num_buildings': len(building_detections),
        'damage_summary': self._create_damage_summary(building_detections),
        'coordinate_scaling_applied': png_image.shape[:2] != tif_image.shape[:2]
    }

    return results

def _create_damage_summary(self, building_detections: List[Dict]) -> Dict:
    """
    Create summary statistics of damage assessment
    """
    if not building_detections:
        return {'total_buildings': 0}

    damage_counts = {cls: 0 for cls in self.damage_classes}
    total_area = 0
    damage_area = {cls: 0 for cls in self.damage_classes}

    for building in building_detections:
        damage_class = building['damage_class']
        area = building['area']

        damage_counts[damage_class] += 1
        damage_area[damage_class] += area
    total_area += area

```

```

summary = {
    'total_buildings': len(building_detections),
    'total_area': total_area,
    'damage_counts': damage_counts,
    'damage_area': damage_area,
    'damage_percentages': {
        cls: (count / len(building_detections) * 100)
        for cls, count in damage_counts.items()
    }
}

return summary

def visualize_results(self, results: Dict, save_path: Optional[str] = None):
    """
    Simplified visualization showing only original image and pipeline results
    """
    fig, axes = plt.subplots(1, 2, figsize=(16, 8))

    tif_image = results['original_tif_image']
    buildings = results['buildings']

    # Original image (clean, no annotations)
    axes[0].imshow(tif_image)
    axes[0].set_title('Original Image')
    axes[0].axis('off')

    # Pipeline results with damage classification overlay
    damage_overlay = tif_image.copy()
    damage_colors = {
        'no-damage': [0, 255, 0],      # Green
        'minor-damage': [255, 255, 0], # Yellow
        'major-damage': [255, 165, 0], # Orange
        'destroyed': [255, 0, 0]       # Red
    }

    for building in buildings:
        x1, y1, x2, y2 = building['bbox']
        color = damage_colors.get(building['damage_class'], [255, 255, 255])
        cv2.rectangle(damage_overlay, (x1, y1), (x2, y2), color, 3)

    axes[1].imshow(damage_overlay)
    axes[1].set_title('Pipeline Results')
    axes[1].axis('off')

    plt.tight_layout()

    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')

    plt.show()

    # Print summary
    self._print_summary(results)

def _print_summary(self, results: Dict):
    """

```

```

Print detailed summary of results
"""
buildings = results['buildings']
summary = results['damage_summary']

print("\n" + "="*50)
print("BUILDING DAMAGE ASSESSMENT SUMMARY")
print("="*50)
print(f"Total buildings detected: {summary['total_buildings']}")

if summary['total_buildings'] > 0:
    avg_detection_conf = np.mean([b['confidence'] for b in buildings])
    print(f"Average YOLO confidence: {avg_detection_conf:.3f}")
    print(f"Total building area: {summary['total_area']} pixels")

print("\nDamage breakdown:")
for damage_class, count in summary['damage_counts'].items():
    if count > 0:
        percentage = summary['damage_percentages'][damage_class]
        avg_damage_conf = np.mean([b['damage_confidence'] for b in buildings
                                    if b['damage_class'] == damage_class])
        print(f"    • {damage_class.upper()}: {count} buildings ({percentage:.1f}%) "
              f"- Avg confidence: {avg_damage_conf:.3f}")

print("\nIndividual building details:")
for building in buildings:
    print(f"    Building #{building['id']}: {building['damage_class']} "
          f"(YOLO: {building['confidence']:.2f}, Damage: {building['damage_confidence']:.2f})")

print("="*50)

def create_yolo_pipeline_from_saved_models(yolo_model_path: str,
                                           classification_model_path: str,
                                           device: str = 'cuda',
                                           confidence_threshold: float = 0.1) -> YOLOBuildingDamageAssessmentPipeline:
    """
    Create YOLO pipeline from saved model checkpoints
    """
    # Build EfficientNet-B3 backbone with ImageNet weights
    from torchvision.models import efficientnet_b3, EfficientNet_B3_Weights
    classification_model = efficientnet_b3(weights=EfficientNet_B3_Weights.IMAGENET1K_V1)

    # Replace the final classifier layer to match 4 classes
    in_feats = classification_model.classifier[1].in_features
    classification_model.classifier[1] = nn.Linear(in_feats, 4)

    # Load your fine-tuned checkpoint (common key variants handled)
    ckpt = torch.load(classification_model_path, map_location=device)
    if isinstance(ckpt, dict) and any(k in ckpt for k in ["state_dict", "model_state_dict"]):
        ckpt = ckpt.get("state_dict", ckpt.get("model_state_dict"))

    if isinstance(ckpt, dict):
        new_ckpt = {}
        for k, v in ckpt.items():
            new_k = k.replace("module.", "")
            new_ckpt[new_k] = v

```



```

        ckpt = new_ckpt

# Load the classification model state
missing, unexpected = classification_model.load_state_dict(ckpt, strict=False)
if missing or unexpected:
    print("Warning: load_state_dict issues:")
    if missing:
        print("  Missing keys:", missing)
    if unexpected:
        print("  Unexpected keys:", unexpected)

# Create pipeline
pipeline = YOLOBuildingDamageAssessmentPipeline(
    yolo_model_path=yolo_model_path,
    classification_model=classification_model,
    device=device,
    confidence_threshold=confidence_threshold,
    crop_size=224
)
return pipeline

# Create YOLO pipeline (adjust paths to your saved models)
pipeline = create_yolo_pipeline_from_saved_models(
    yolo_model_path=r'C:\Users\sajja\Desktop\Dissertation\Object Detection\building_detection_yolov8s_50epochs\weights\best.pt',
    classification_model_path=r'C:\Users\sajja\Desktop\Dissertation\Classification Models\Fine Tuning\Efficientnet\efficientnet_stage3.pth',
    device='cuda' if torch.cuda.is_available() else 'cpu',
    confidence_threshold=0.1
)

# Load a .tif image using tifffile (handles complex .tif formats better)
image_path = r'C:\Users\sajja\Desktop\Dissertation\geotiffs\test\images\hurricane-harvey_00000485_post_disaster.tif'
image = imread(image_path)

# If dtype is not uint8, scale
if image.dtype != np.uint8:
    image = image.astype(np.float32)
    image = (image / image.max() * 255).clip(0, 255).astype(np.uint8)

results = pipeline.process_image(image)

# Visualize results
pipeline.visualize_results(results)

```

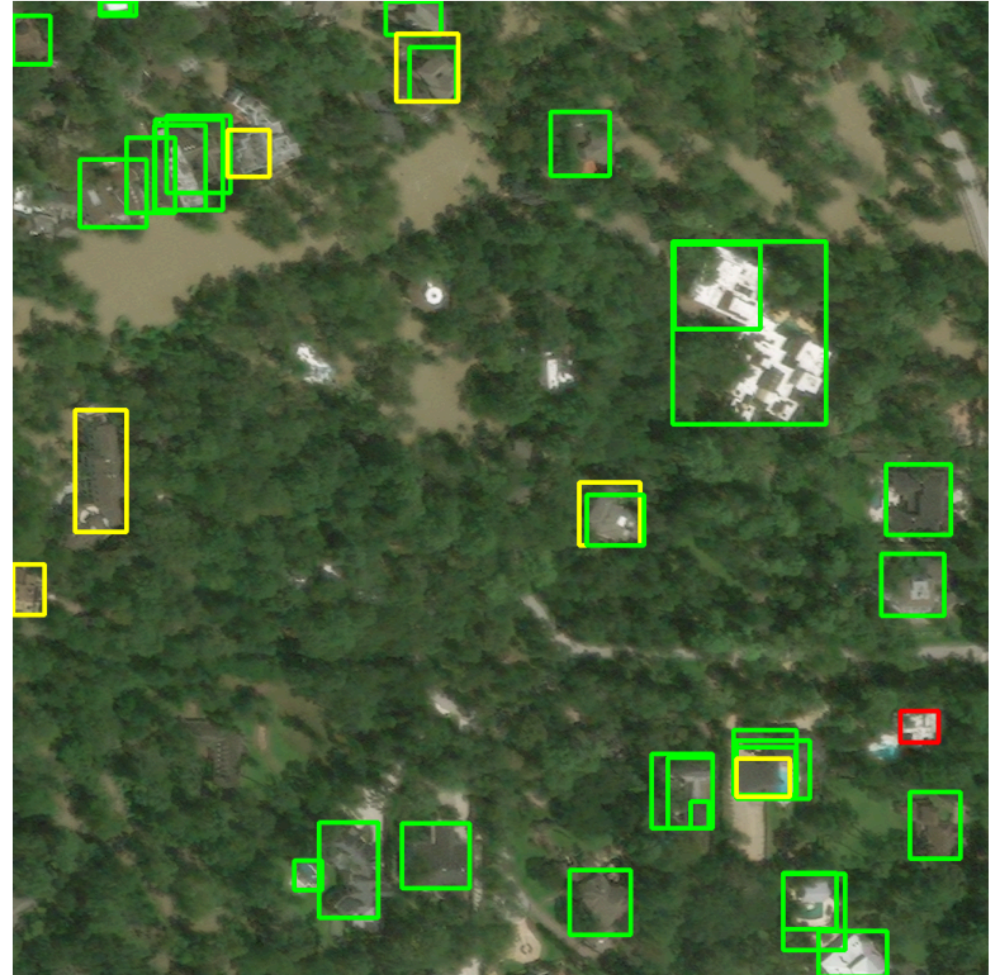
↩ Converting .tif to .png for YOLO detection...  
 Stage 1: Building detection using YOLO on .png

0: 1024x1024 37 buildings, 28.7ms  
 Speed: 4.4ms preprocess, 28.7ms inference, 2.8ms postprocess per image at shape (1, 3, 1024, 1024)  
 Extracting crops for 37 detected buildings from original .tif  
 Stage 2: Classifying damage for 37 buildings

Original Image



Pipeline Results



=====

#### BUILDING DAMAGE ASSESSMENT SUMMARY

=====

Total buildings detected: 37  
 Average YOLO confidence: 0.240  
 Total building area: 165874 pixels

#### Damage breakdown:

- NO-DAMAGE: 30 buildings (81.1%) - Avg confidence: 0.897

- MINOR-DAMAGE: 6 buildings (16.2%) - Avg confidence: 0.721
- DESTROYED: 1 buildings (2.7%) - Avg confidence: 0.465

## Individual building details:

Building #0: no-damage (YOLO: 0.65, Damage: 1.00)  
Building #1: no-damage (YOLO: 0.58, Damage: 0.99)  
Building #2: no-damage (YOLO: 0.58, Damage: 0.89)  
Building #3: minor-damage (YOLO: 0.56, Damage: 0.81)  
Building #4: no-damage (YOLO: 0.52, Damage: 0.96)  
Building #5: no-damage (YOLO: 0.51, Damage: 0.96)  
Building #6: minor-damage (YOLO: 0.51, Damage: 0.57)  
Building #7: no-damage (YOLO: 0.43, Damage: 0.88)  
Building #8: destroyed (YOLO: 0.28, Damage: 0.46)  
Building #9: no-damage (YOLO: 0.28, Damage: 0.70)  
Building #10: no-damage (YOLO: 0.26, Damage: 0.90)  
Building #11: no-damage (YOLO: 0.23, Damage: 0.72)  
Building #12: no-damage (YOLO: 0.23, Damage: 0.96)  
Building #13: no-damage (YOLO: 0.21, Damage: 0.98)  
Building #14: no-damage (YOLO: 0.20, Damage: 1.00)  
Building #15: no-damage (YOLO: 0.18, Damage: 0.78)  
Building #16: minor-damage (YOLO: 0.17, Damage: 0.60)  
Building #17: no-damage (YOLO: 0.17, Damage: 0.88)  
Building #18: no-damage (YOLO: 0.16, Damage: 0.89)  
Building #19: no-damage (YOLO: 0.15, Damage: 0.94)  
Building #20: no-damage (YOLO: 0.15, Damage: 0.92)  
Building #21: no-damage (YOLO: 0.14, Damage: 0.88)  
Building #22: no-damage (YOLO: 0.14, Damage: 0.89)  
Building #23: no-damage (YOLO: 0.13, Damage: 0.97)  
Building #24: no-damage (YOLO: 0.13, Damage: 0.93)  
Building #25: minor-damage (YOLO: 0.12, Damage: 0.97)  
Building #26: no-damage (YOLO: 0.12, Damage: 0.59)  
Building #27: no-damage (YOLO: 0.12, Damage: 1.00)  
Building #28: no-damage (YOLO: 0.11, Damage: 0.99)  
Building #29: no-damage (YOLO: 0.11, Damage: 0.82)  
Building #30: no-damage (YOLO: 0.11, Damage: 0.54)  
Building #31: minor-damage (YOLO: 0.11, Damage: 0.77)  
Building #32: no-damage (YOLO: 0.11, Damage: 1.00)  
Building #33: no-damage (YOLO: 0.10, Damage: 1.00)  
Building #34: no-damage (YOLO: 0.10, Damage: 0.95)  
Building #35: no-damage (YOLO: 0.10, Damage: 1.00)  
Building #36: minor-damage (YOLO: 0.10, Damage: 0.62)

=====

```
# Create YOLO pipeline (adjust paths to your saved models)
pipeline = create_yolo_pipeline_from_saved_models(
    yolo_model_path="/content/drive/MyDrive/Building Damage Assessment/Object_Detection_Model/building_detection_yolov8s_50epochs/weights/best.pt",
    classification_model_path="/content/drive/MyDrive/Building Damage Assessment/Best_Classification_Models/efficientnet_stage3.pth",
    device='cuda' if torch.cuda.is_available() else 'cpu',
    confidence_threshold=0.1
)

# Load a .tif image using tifffile (handles complex .tif formats better)
image_path = "/content/drive/MyDrive/Building Damage Assessment/Image Example used in Pipeline/hurricane-harvey_00000485_post_disaster.tif"
image = imread(image_path)

# If dtype is not uint8, scale
if image.dtype != np.uint8:
    image = image.astype(np.float32)
    image = (image / image.max() * 255).clip(0, 255).astype(np.uint8)

results = pipeline.process_image(image)

# Visualize results
pipeline.visualize_results(results)
```